# CS528 Homework 4: Web Server and Forbidden-Country Service on Compute Engine VMs

Oladunni Ajayi (oajayi@bu.edu)
Code: https://github.com/jollofbytes/CS528hw4
GCP project: `solar-attic-508721-n0` Bucket: `gs://cs528hww2` (HW2 files under `CS528-HW_2-output-x2Yenv/`)

## 1. Overview

HW3's two services were moved from Cloud Run and my laptop onto three Compute Engine VMs. Each VM runs as its own service account with only the permissions it needs, and each service starts automatically when its VM is created.

```
hw4-client (Ubuntu 24.04, e2-small)          course http-client
       |  HTTP GET/POST, port 80
       v
hw4-server (Debian 12, e2-micro, static IP 104.198.201.20, SA hw4-server-sa)
       |-- reads files ------------------> gs://cs528hww2/CS528-HW_2-output-x2Yenv/
       |-- 404/501/400 structured logs --> Cloud Logging (WARNING / ERROR / CRITICAL)
       '-- forbidden country -----------> Pub/Sub topic forbidden-requests
                                                | subscription forbidden-requests-sub
                                                v
hw4-forbidden (Debian 12, e2-micro, SA hw4-forbidden-sa) --> prints to stdout
```

| File (GitHub) | Purpose |
|---|---|
| HW4.py | Service 1: the web server (Python http.server) |
| forbidden_service.py | Service 2: Pub/Sub subscriber that prints forbidden requests |
| startup-server.sh, startup-forbidden.sh | VM startup scripts: install packages, run each service under systemd |
| deploy.sh | Step 9: creates all resources and both VMs in one command; also deletes them |
| startup-client.sh, http-client.pyc.b64 | Client VM setup (the course http-client, see Step 4) |
| stress.sh | Step 8: runs N concurrent clients for 3 minutes and counts responses |

## 2. Configuring and running the apps (Step 9)

Everything is created by one script. Prerequisites: the gcloud CLI, logged in (`gcloud auth login`) as an Owner/Editor of the project, and a bucket containing the HW2 files. From a clone of the repo:

```
bash deploy.sh              # create everything and start both services
bash deploy.sh client       # same, plus the client VM
bash deploy.sh delete       # delete all HW4 VMs
bash deploy.sh delete-all   # also delete the static IP, firewall rule and service accounts

# Another project / bucket, without editing the file:
PROJECT_ID=my-project BUCKET=my-bucket bash deploy.sh
```

**What deploy.sh does, in order** (every step is skipped if the resource already exists, so it is safe to rerun):

1. Sets the project and enables the Compute Engine, Pub/Sub and Cloud Logging APIs.
2. Creates the Pub/Sub topic `forbidden-requests` and subscription `forbidden-requests-sub`.

3. Creates two service accounts and grants least-privilege roles (table below).
4. Reserves the static IP `hw4-server-ip` and opens port 80 with a firewall rule that applies only to VMs tagged `hw4-server`.
5. Creates the two VMs. The code and startup script are attached to each VM as metadata (`--metadata-from-file`), so no key files, git clone or extra bucket access is needed. On boot, the startup script installs the Python packages in a virtualenv, pulls the code from the metadata server and starts it as a **systemd** service (`Restart=always`), so the service also comes back after a reboot or crash.

### Service accounts and permissions

| VM | Service account | Roles (and where granted) |
|---|---|---|
| hw4-server | hw4-server-sa | storage.objectViewer on the bucket only<br>pubsub.publisher on the topic only<br>logging.logWriter on the project |
| hw4-forbidden | hw4-forbidden-sa | pubsub.subscriber on the subscription only |
| hw4-client | none | No service account: it only sends HTTP requests |

The roles are granted on the individual bucket, topic and subscription rather than project-wide, so the server cannot write to the bucket or read the subscription, and the forbidden service cannot touch storage or logs. (For the stress test only, I also gave hw4-server-sa `monitoring.metricWriter` and installed the Ops Agent so memory usage would be reported.)

I tested the script end to end: `bash deploy.sh delete`, then `bash deploy.sh`. About 3 minutes later both services were up, and a forbidden request was printed by the new hw4-forbidden VM.

## 3. Step 1: the web server

`HW4.py` uses Python's `ThreadingHTTPServer` (one thread per request) listening on port 80. It keeps the HW3 logic:

| Request | Response |
|---|---|
| GET /folder/N.html (file in the URL path) | 200 + file contents from the bucket |
| POST with the file name in the body (JSON `{"filename": ...}`, form field, or raw text) | 200 + file contents |
| File not in the bucket | 404, logged with severity WARNING |
| PUT, DELETE, HEAD, CONNECT, OPTIONS, TRACE, PATCH | 501, logged with severity ERROR |
| Header `X-country` is North Korea, Iran, Cuba, Myanmar, Iraq, Libya, Sudan, Zimbabwe or Syria | 400, logged with severity CRITICAL and published to Pub/Sub |

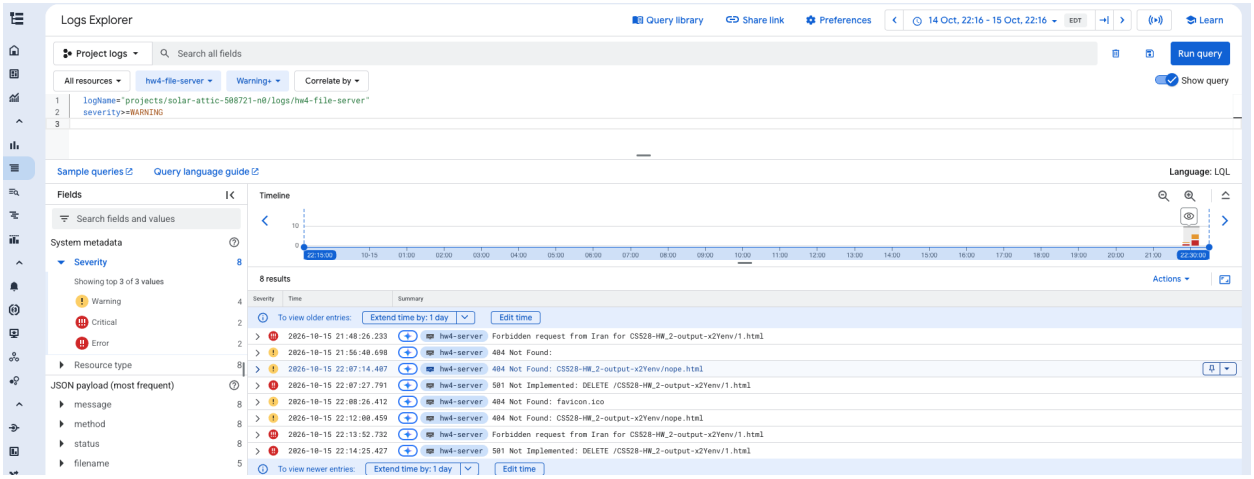
Logs are written with the `google-cloud-logging` library as structured entries (`log_struct`) to the log `hw4-file-server`, with fields such as status, method, filename and country.

## 4. Steps 2, 3 and 7: Cloud Logging

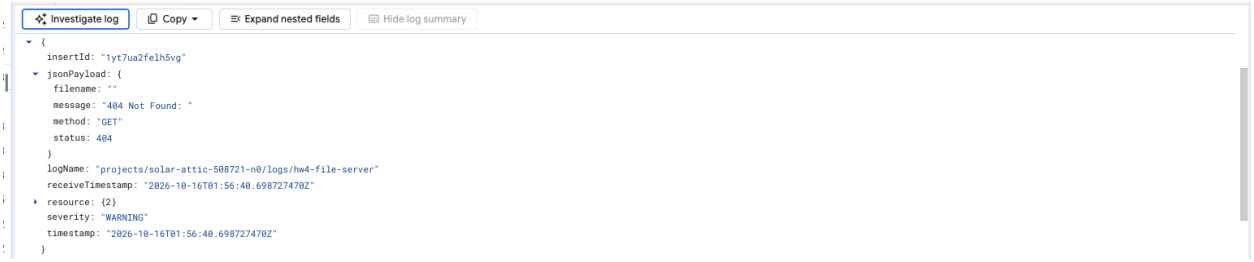
Logs Explorer query:

```
logName="projects/solar-attic-508721-n0/logs/hw4-file-server" AND
```

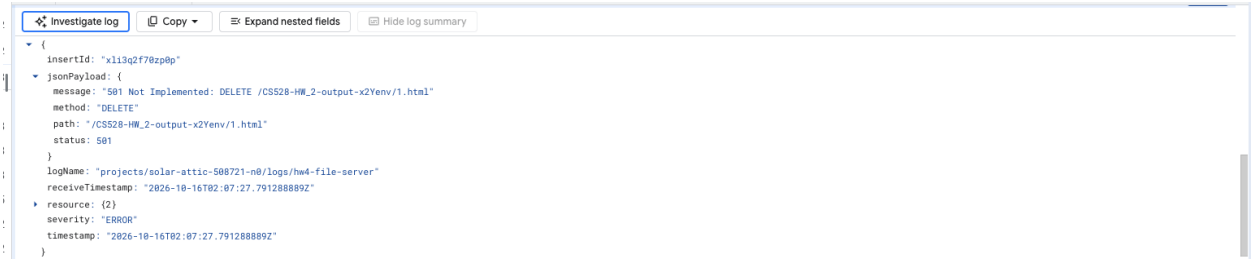
```
severity>=WARNING
```




Erroneous requests: WARNING for 404s, ERROR for 501s, CRITICAL for forbidden-country requests.




WARNING entry (404). This one was a request for the bare path "/", so filename is empty.




ERROR entry (501 for DELETE), with method and path as structured fields.




CRITICAL entry (400 for a request from Iran), with country, filename, method and status.

# 5. Step 4: 100 requests from a client VM

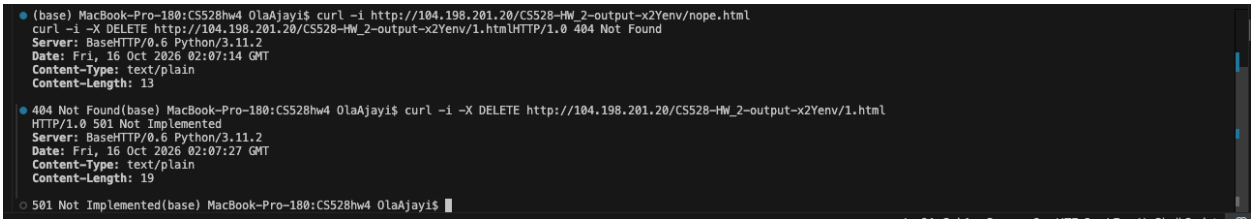
The provided http-client is a macOS build. It is a PyInstaller-packaged Python 3.12 program, so I extracted its bytecode (`http-client.pyc`, unchanged) and run it with Python 3.12 on the hw4-client

VM (Ubuntu 24.04), which is separate from the server VM. Command, run from my laptop on the client VM:

```
gcloud compute ssh hw4-client --zone=us-central1-a --command \
  "http-client -d 104.198.201.20 -p 80 -b none -w CS528-HW_2-output-x2Yenv -n 100 -i 10000 -v \
   | grep -E '^[0-9]{3} '"
```

Result: **98 x 200 OK** and **2 x 400 Bad Request**. The 400s are requests the client randomly tagged with a forbidden country. There were no 404s, because every requested file exists.

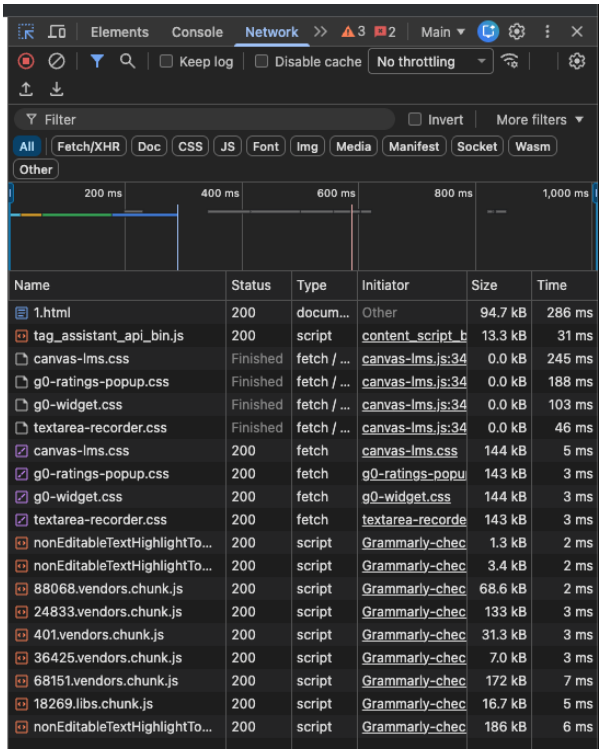
## 6. Step 5: curl (404 and 501)

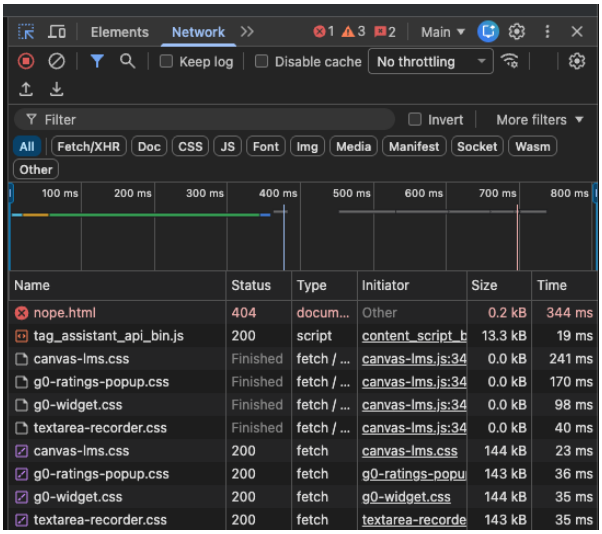



`curl -i .../nope.html` returns 404 Not Found; `curl -i -X DELETE .../1.html` returns 501 Not Implemented.

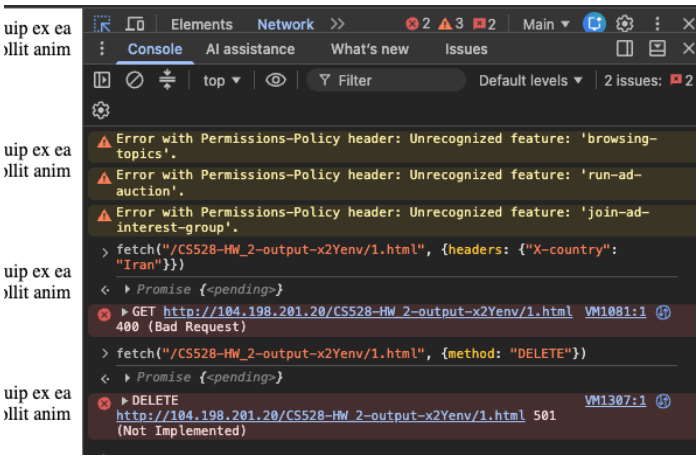
## 7. Step 6: browser

Chrome with DevTools open. The 200 and 404 are normal page loads. Browsers cannot change the method or add headers from the address bar, so the 400 and 501 were sent with `fetch()` from the DevTools console on the server's page.




Left: 200 for 1.html. Right: 404 for nope.html. Other rows in the list are browser extensions.

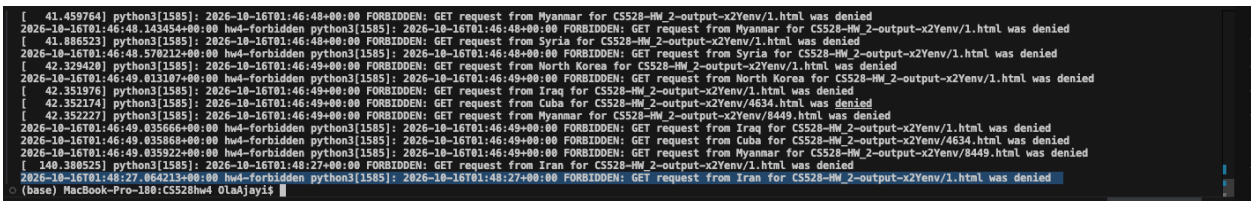



400 for a request with X-country: Iran, and 501 for DELETE.

## 8. Step 7: forbidden-country service on a third VM

The server publishes each forbidden request to Pub/Sub. `forbidden_service.py` on hw4-forbidden subscribes and prints a message to stdout. Because it runs on a VM, the Google libraries authenticate automatically as the VM's service account, so the HW3 impersonation code was removed. systemd also copies stdout to the serial console, so the output can be read without SSH:

```
gcloud compute instances get-serial-port-output hw4-forbidden \
    --zone=us-central1-a | grep FORBIDDEN
```




Service 2 output. The last line is my Iran test request. The earlier lines are forbidden requests from HW3 testing that were still queued in the subscription and were delivered when the service started.

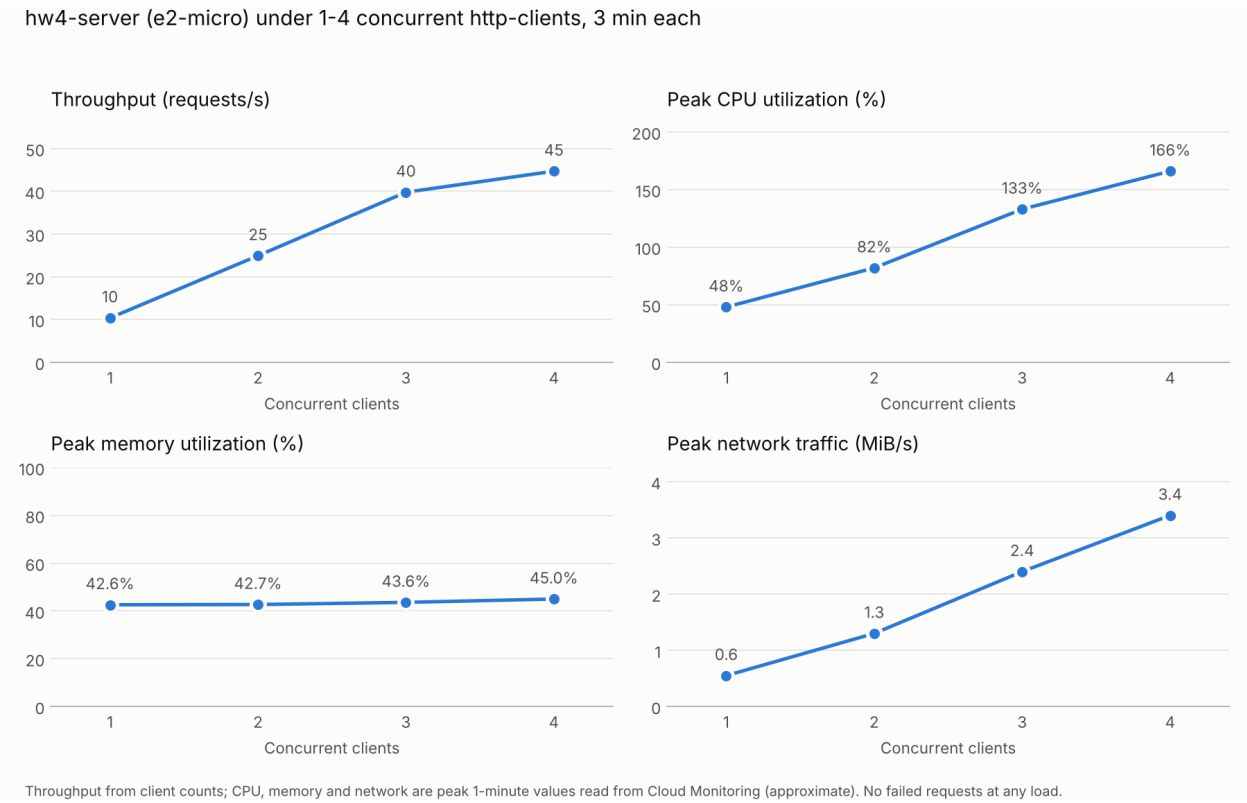
## 9. Step 8: stress test

The server ran on **e2-micro**, the smallest general-purpose machine type. `stress.sh` starts N copies of the http-client at the same time on the client VM, each requesting random files nonstop for 3 minutes, and counts every response. I ran N = 1, 2, 3, 4 with about a minute between runs. A failure would be anything other than 200 or 400: a timeout, a connection error or a 5xx.

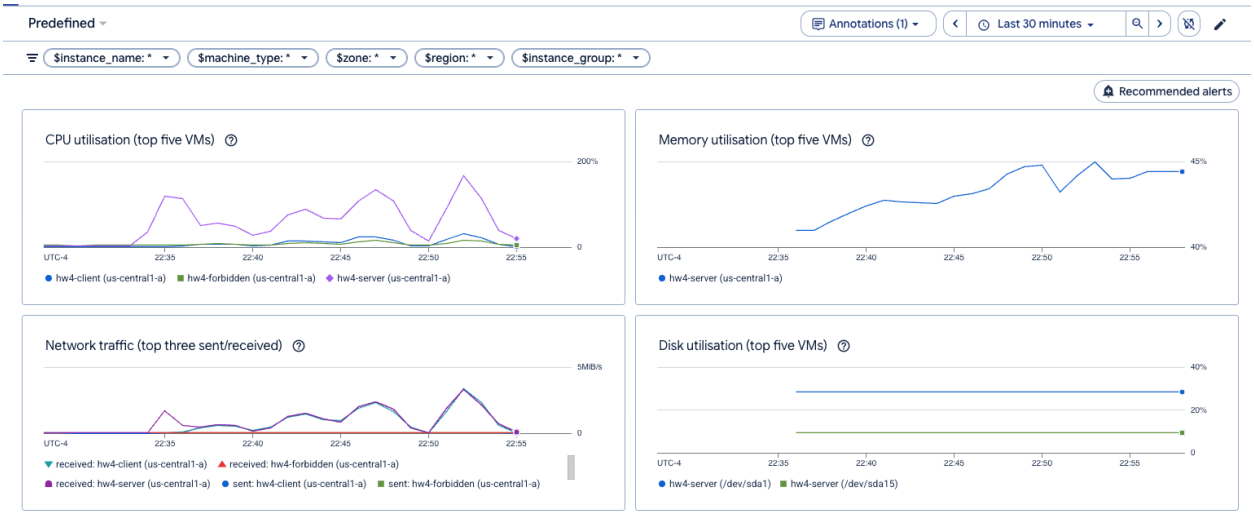
| Clients | Time (UTC) | 200 OK | 400 | Failures | Requests/s | Per client |
|---|---|---|---|---|---|---|
| 1 | 02:36-02:39 | 1,782 | 87 | 0 | 10.4 | 10.4 |
| 2 | 02:40-02:43 | 4,296 | 192 | 0 | 24.9 | 12.5 |
| 3 | 02:44-02:47 | 6,833 | 329 | 0 | 39.8 | 13.3 |
| 4 | 02:50-02:53 | 7,696 | 352 | 0 | 44.7 | 11.2 |

**Finding:** the server never returned an error, even with 4 concurrent clients, so it was not saturated to the point of failing. However, it was close to its limit at 4 clients: total throughput rose almost linearly from 1 to 3 clients (10 to 40 req/s) but only to 45 req/s at 4, and each client's rate dropped. Requests

were starting to queue, so responses got slower rather than failing. As the assignment asks when 4 clients do not saturate the server, the peak resource usage is reported below.




Throughput is exact (from client counts). CPU, memory and network are the peak 1-minute values for hw4-server from Cloud Monitoring (Ops Agent installed for memory), so they are approximate.




Compute Engine Observability for the test window (EDT). The four bumps are the 1-4 client runs. The first spike, at about 22:34, is the Ops Agent being installed, not part of the test.

## How CPU, memory and bandwidth scale, and why

**CPU** scales roughly linearly with load (about 48% to 166%). Every request costs CPU: fetching the file from Cloud Storage over HTTPS, sending it back, and for errors writing a log entry or publishing to

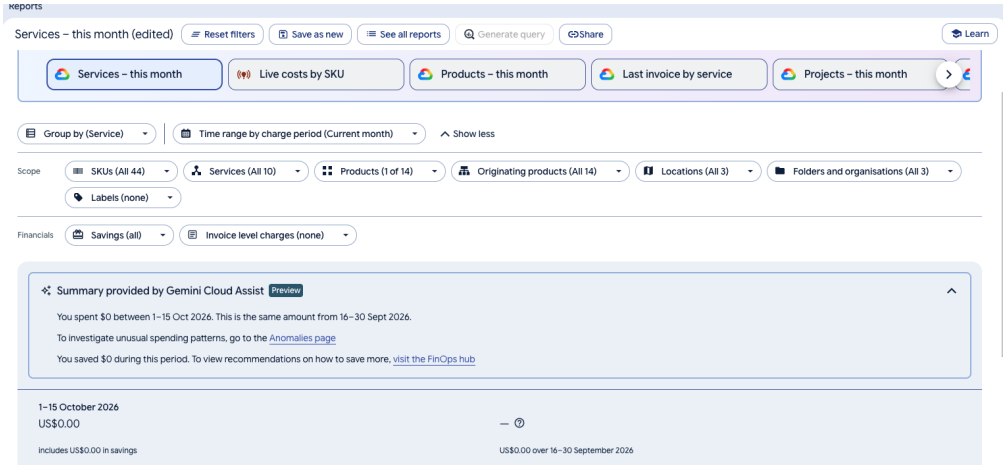
Pub/Sub. e2-micro is a shared-core machine that can burst above its baseline share, which is why the chart exceeds 100%. CPU is the resource closest to its limit, and it explains the throughput levelling off between 3 and 4 clients.

**Network** scales with throughput (about 0.6 to 3.4 MiB/s). Each request moves one HTML file in from the bucket and out to the client, so bytes per second track requests per second.

**Memory** stays nearly flat (about 43% to 45% of roughly 1 GB). The server holds no per-request state beyond a short-lived thread and one file in memory, so more clients add very little. Most of the memory in use is the OS, Python and the monitoring agent.

## Cost

Billing showed **$0.00** for the month to date when checked right after the test. Compute Engine list prices (e2-micro about $0.008/h, which is also covered by the free tier in us-central1; e2-small about $0.017/h) and roughly 1 GB of network traffic put the real cost of the stress test at a few cents (well under $0.10). To limit spending, I used the smallest machine types, capped each run at 3 minutes and 4 clients, and deleted the VMs afterwards with `bash deploy.sh delete`.



Cloud Billing report, Compute Engine, 1-15 October 2026.